ACVP KAS ECC SP800-56Ar3 JSON Specification

Russell Hammett
HII Technical Solutions Division
302 Sentinel Drive, Suite #300, Annapolis Junction, MD 20701

January 31, 2020

National Institute of
Standards and Technology
U.S. Department of Commerce

Abstract

This document defines the JSON schema for testing SP800-56Ar3 KAS ECC implementations with the ACVP specification.

Keywords

The following are keywords to be used by search engines and document catalogues.

ACVP; cryptography

Foreword

The Information Technology Laboratory (ITL) at the National Institute of Standards and Technology (NIST) promotes the U.S. economy and public welfare by providing technical leadership for the Nation's measurement and standards infrastructure. ITL develops tests, test methods, reference data, proof of concept implementations, and technical analyses to advance the development and productive use of information technology. ITL's responsibilities include the development of management, administrative, technical, and physical standards and guidelines for the cost-effective security and privacy of other than national security-related information in federal information systems. The Special Publication 800-series reports on ITL's research, guidelines, and outreach efforts in information system security, and its collaborative activities with industry, government, and academic organizations.

Audience

This document is intended for the users and developers of ACVP.

Conventions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 of [RFC 2119] and [RFC 8174] when, and only when, they appear in all capitals, as shown here.

Acknowledgements

This document is produced by the Security Testing, Validation and Measurement group under the Automated Cryptographic Validation Testing (ACVT) program.

Executive Summary

The Automated Crypto Validation Protocol (ACVP) defines a mechanism to automatically verify the cryptographic implementation of a software or hardware crypto module. The ACVP specification defines how a crypto module communicates with an ACVP server, including crypto

capabilities negotiation, session management, authentication, vector processing and more. The ACVP specification does not define algorithm specific JSON constructs for performing the crypto validation. A series of ACVP sub-specifications define the constructs for testing individual crypto algorithms. Each sub-specification addresses a specific class of crypto algorithms. This sub-specification defines the JSON constructs for testing SP800-56Ar3 KAS ECC implementations using ACVP.

Disclaimer

Any mention of commercial products or reference to commercial organizations is for information only; it does not imply recommendation or endorsement by NIST, nor does it imply that the products mentioned are necessarily the best available for the purpose.

Additional Information

For additional information on NIST's Cybersecurity programs, projects and publications, visit the Computer Security Resource Center. Information on other efforts at NIST and in the Information Technology Laboratory (ITL) is also available.

Feedback

Feedback on this publication is welcome, and can be sent to: code-signing@nist.gov.

1. Introduction

The Automated Crypto Validation Protocol (ACVP) defines a mechanism to automatically verify the cryptographic implementation of a software or hardware crypto module. The ACVP specification defines how a crypto module communicates with an ACVP server, including crypto capabilities negotiation, session management, authentication, vector processing and more. The ACVP specification does not define algorithm specific JSON constructs for performing the crypto validation. A series of ACVP sub-specifications define the constructs for testing individual crypto algorithms. Each sub-specification addresses a specific class of crypto algorithms. This sub-specification defines the JSON constructs for testing SP800-56Ar3 KAS ECC implementations using ACVP.

2. Supported KAS-ECCs

The following key derivation functions **MAY** be advertised by the ACVP compliant cryptographic module:

- KAS-ECC / null / Sp800-56Ar3
- KAS-ECC / CDH-Component / Sp800-56Ar3

3. Test Types and Test Coverage

The ACVP server performs a set of tests on the KAS protocol in order to assess the correctness and robustness of the implementation. A typical ACVP validation session **SHALL** require multiple tests to be performed for every supported permutation of KAS capabilities. This section describes the design of the tests used to validate implementations of KAS algorithms.

3.1. Test Types

There are two test types for KAS testing:

- "AFT" — Algorithm Function Test. In the AFT test mode, the IUT **SHALL** act as a party in the Key Agreement with the ACVP server. The server **SHALL** generate and provide all necessary information for the IUT to perform a successful key agreement; both the server and IUT **MAY** act as party U/V, as well as recipient/provider to key confirmation.
- "VAL" — Validation Test. In the VAL test mode, The ACVP server **MUST** generate a complete (from both party U and party V's perspectives) key agreement, and expects the IUT to be able to determine if that agreement is valid. Various types of errors **MUST** be introduced in varying portions of the key agreement process (changed DKM, changed key, changed hash digest, etc), that the IUT **MUST** be able to detect and report on.

3.2. Test Coverage

The tests described in this document have the intention of ensuring an implementation is conformant to [SP 800-56A Rev. 3].

3.2.1. KAS-ECC Requirements Covered

- SP 800-56Ar3 — 5.1 Cryptographic Hash Functions. SHA1, SHA2, and SHA3 hash functions **SHALL** be available for the various pieces of KAS requiring use of a hash function; such as approved MACs and OneStep KDF.
- SP 800-56Ar3 — 5.2 Message Authentication Code (MAC) Algorithms. AES-CMAC, HMAC, and KMAC algorithms **SHALL** be available for testing under KDFs and KC as the specification states.
- SP 800-56Ar3 — 5.3 Random Number Generation. Though random values are used, the testing of the construction of those random values **SHALL NOT** be in scope of ACVP testing.
- SP 800-56Ar3 — 5.4 Nonces. Though nonces are used, the testing of the construction of those nonces **SHALL NOT** be in scope of ACVP testing.
- SP 800-56Ar3 — 5.5 Domain Parameters. Domain Parameters **SHALL** be used in the testing of KAS as per the specification, though the generation of those parameters is outside the scope of testing.
- SP 800-56Ar3 — 5.6 Key-Pair Generation. Each KAS scheme from one or both parties utilizes a key pair for arriving at a shared secret, and deriving a key. Though a key pair(s)

are utilized in ACVP testing, the testing of the generation of said key pairs is outside the scope of this testing.

- SP 800-56Ar3 — 5.7 DLC Primitives. Diffie Hellman and MQV **SHALL** be tested under their respective KAS schemes.
- SP 800-56Ar3 — 5.8 Key-Derivation Methods for Key-Establishment Schemes. The ACVP server **SHALL** make various KDFs available for testing. The KDFs covered under ACVP server testing **SHALL** include the KDFs specified in SP800-56B, SP800-56C, SP800-108, and SP800-135 (where applicable).
- SP 800-56Ar3 — 5.9 KeyConfirmation. The ACVP server **SHALL** support key confirmation for applicable KAS and KTS schemes.
- SP 800-56Ar3 — 6 Key Agreement Schemes. The ACVP server **SHALL** support testing for all KAS schemes specified in the SP800-56Ar3 document.
- SP 800-56Cr1 — 4 One-Step Key Derivation. One-Step Key Derivation testing **SHALL** be supported by the ACVP server. FixedInfo construction is covered within the ACVP specification, and can be tailored to the IUTs needs. ASN.1 format of fixedInfo construction (currently) is NOT supported.
- SP 800-56Cr1 — 5 Two-Step Key Derivation. Two-Step Key Derivation testing **SHALL** be supported by the ACVP server. FixedInfo construction is covered within the ACVP specification, and can be tailored to the IUTs needs. ASN.1 format of fixedInfo construction (currently) is NOT supported.
- SP 800-108 — 4 Pseudorandom Function (PRF). All iterations of the KDF described in SP800-108 use a separate PRF. All implementations of the PRF **SHALL** be available for testing through the ACVP server generated tests.
- SP 800-108 — 5 Key Derivation Functions (KDF). The three implementations of KDFs in SP800-108 **SHALL** be available for testing through the ACVP Server.

3.2.2. KAS-ECC Requirements Not Covered

- SP 800-56Ar3 — 4.3 DLC-based Key-Transport Process. KeyWrapping is not incorporated into ACVP testing.
- SP 800-56Ar3 — 5.3 Random Number Generation. Though random values are used, the testing of the construction of those random values **SHALL NOT** be in scope of ACVP testing.
- SP 800-56Ar3 — 5.4 Nonces. Though nonces are used, the testing of the construction of those nonces **SHALL NOT** be in scope of ACVP testing.
- SP 800-56Ar3 — 5.5 Domain Parameters. Domain Parameters **SHALL** be used in the testing of KAS as per the specification, though the generation of those parameters is outside the scope of testing.
- SP 800-56Ar3 — 5.6 Key-Pair Generation. Each KAS scheme from one or both parties utilizes a key pair for arriving at a shared secret, and deriving a key. Though a key pair(s) are utilized in ACVP testing, the testing of the generation of said key pairs is outside the scope of this testing.

- SP 800-56Ar3 — 5.6.2 Required Assurances. IUT assurance testing is outside the scope of ACVP testing.
- SP 800-56Ar3 — 5.6.2 Key Pair Management. Testing the IUT's management of Key Pairs is outside the scope of ACVP testing.
- SP 800-56Ar3 — 5.8.1.2 The ASN.1 Format for FixedInfo. The ACVP server (currently) **SHALL NOT** support the testing of this format of fixed info.
- SP 800-56Ar3 — 7 Rationale for Selecting a Specific Scheme. There is no testing associated with the IUT's choice of selecting a specific scheme.
- SP 800-56Ar3 — 8 Key Recovery. Key Recovery **SHALL NOT** be within the scope of ACVP testing.
- SP 800-56Cr1 — 4 One-Step Key Derivation. ASN.1 format of fixedInfo construction (currently) is NOT supported.
- SP 800-56Cr1 — 5 Two-Step Key Derivation. ASN.1 format of fixedInfo construction (currently) is NOT supported.
- SP 800-56Cr1 — 7 Selecting Hash Functions and MAC Algorithms. The process that goes into the selection of Hash functions and MAC algorithms **SHALL NOT** be in scope of ACVP testing, though the ACVP server **SHALL** support all indicated Hash and MAC functions.
- SP 800-56Cr1 — 7 Selecting Hash Functions and MAC Algorithms. The process that goes into the selection of Hash functions and MAC algorithms **SHALL NOT** be in scope of ACVP testing, though the ACVP server **SHALL** support all indicated Hash and MAC functions.

4. Capabilities Registration

ACVP requires crypto modules to register their capabilities. This allows the crypto module to advertise support for specific algorithms, notifying the ACVP server which algorithms need test vectors generated for the validation process. This section describes the constructs for advertising support of KAS ECC algorithms to the ACVP server.

The algorithm capabilities **MUST** be advertised as JSON objects within the 'algorithms' value of the ACVP registration message. The 'algorithms' value is an array, where each array element is an individual JSON object defined in this section. The 'algorithms' value is part of the 'capability_exchange' element of the ACVP JSON registration message. See the ACVP specification [ACVP] for more details on the registration message.

4.1. Prerequisites

Each algorithm implementation **MAY** rely on other cryptographic primitives. For example, RSA Signature algorithms depend on an underlying hash function. Each of these underlying algorithm primitives must be validated, either separately or as part of the same submission. ACVP provides a mechanism for specifying the required prerequisites:

Prerequisites, if applicable, **MUST** be submitted in the registration as the `prereqVals` JSON property array inside each element of the `algorithms` array. Each element in the `prereqVals` array **MUST** contain the following properties

Table 1 — Prerequisite Properties

JSON Property	Description	JSON Type
algorithm	a prerequisite algorithm	string
valValue	algorithm validation number	string

A "valValue" of "same" **SHALL** be used to indicate that the prerequisite is being met by a different algorithm in the capability exchange in the same registration.

An example description of prerequisites within a single algorithm capability exchange looks like this

```
"prereqVals":
[
  {
    "algorithm": "Alg1",
    "valValue": "Val-1234"
  },
  {
    "algorithm": "Alg2",
    "valValue": "same"
  }
```

]

Figure 1

4.2. Prerequisite Algorithms

Some algorithm implementations rely on other cryptographic primitives. For example, IKEv2 uses an underlying SHA algorithm. Each of these underlying algorithm primitives must be validated, either separately or as part of the same submission. ACVP provides a mechanism for specifying the required prerequisites:

Table 2 — Prerequisite Algorithms JSON Values

JSON Value	Description	JSON Type	Valid Values	Optional
algorithm	a prerequisite algorithm	value	CMAC, DRBG, ECDSA, HMAC, KMAC, SHA, SP800-108	No
valValue	algorithm validation number	value	actual number or “same”	No
prereqAlgVal	prerequistie algorithm validation	object with algorithm and valValue properties	see above	Yes

KAS has conditional prerequisite algorithms, depending on the capabilities registered:

Table 3 — Prerequisite requirement conditions

Prerequisite Algorithm	Condition
DRBG	Always **REQUIRED**
SHA	Always **REQUIRED**
ECDSA	If the implementation supports fullVal (see Section 4.4), then ECDSA keyVer validation is **REQUIRED**. If the implementation supports keyPairGen (see Section 4.4), then ECDSA keyGen and ECDSA keyVer validation are **REQUIRED**.
CMAC	CMAC validation **REQUIRED** when IUT is performing KeyConfirmation (KC) or a KDF and utilizing CMAC.
HMAC	HMAC validation **REQUIRED** when IUT is performing KeyConfirmation (KC) or a KDF and utilizing HMAC.

Prerequisite Algorithm	Condition
KMAC	KMAC validation **REQUIRED** when IUT is performing KeyConfirmation (KC) or a KDF and utilizing KMAC.

4.3. Algorithm Capabilities JSON Values

Each algorithm capability advertised is a self-contained JSON object using the following values.

Table 4 — Capabilities JSON Values

JSON Value	Description	JSON Type	Valid Values	Optional
algorithm	The algorithm under test	value	KAS-ECC	No
revision	The algorithm testing revision to use.	value	“Sp800-56Ar3”	No
prereqVals	Prerequisite algorithm validations	array of prereqAlgVal objects	See Section 4.2	No
function	Type of function supported	array	See Section 4.4	Yes
iutId	The identifier of the IUT.	hex		No
scheme	Array of supported key agreement schemes each having their own capabilities	object	See Section 4.5.1	No
domainParameterGenerationMethods	Array of IUT supported domain parameter generation methods.	array	P-224, P-256, P-384, P-521, K-233, K-283, K-409, K-571, B-233, B-283,	No

JSON Value	Description	JSON Type	Valid Values	Optional
			B-409, B-571	

Note: Some optional values are **REQUIRED** depending on the algorithm. Failure to provide these values will result in the ACVP server returning an error to the ACVP client during registration.

4.4. Supported KAS ECC Functions

The following function types **MAY** be advertised by the ACVP compliant crypto module:

- keyPairGen — IUT can perform keypair generation.
- partialVal — IUT can perform partial public key validation ([SP800-56Ar3] section 5.6.2.3).
- fullVal — IUT can perform full public key validation ([SP 800-56A Rev. 3] section 5.6.2.3).

4.5. KAS ECC Schemes

All other scheme capabilities are advertised as a self-contained JSON object using the following values. Note that AT LEAST one valid scheme must be registered.

4.5.1. KAS ECC Scheme Capabilities JSON Values

- ephemeralUnified — keyConfirmation not supported.
- fullMqv
- fullUnified
- onePassDh — Can only provide unilateral key confirmation party V to party U.
- onePassMqv
- onePassUnified
- staticUnified

Table 5 — Capabilities JSON Values

JSON Value	Description	JSON Type	Valid Values	Optional
kasRole	Roles supported for key agreement	array	initiator and/ or responder	No
kdfMethods	The KDF methods to use when testing KAS schemes.	object	Section 4.5. 1.1	No
keyConfirmationMethod	The KeyCnfirmation capabilities (when supported) for the scheme.	object	Section 4.5. 1.2	Yes

JSON Value	Description	JSON Type	Valid Values	Optional
	The length of the key to derive (using a KDF) or transport (using a KTS scheme). This value should be large enough to accommodate the key length used for the mac algorithms in use for key confirmation, ideally the maximum value the IUT can support with their KAS/KTS implementation. Maximum value (for testing purposes) is 1024.	integer	128 minimum without KC, 136 minimum with KC, maximum 1024.	No

4.5.1.1. Supported Kdf Methods

Note that AT LEAST one KDF Method is required for KAS schemes. The following **MAY** be advertised by the ACVP compliant crypto module:

Table 6 — KDF Options

JSON Value	Description	JSON Type	Valid Values	Optional
oneStepKdf	Indicates the IUT will be testing key derivation using the SP800-56Cr1 OneStepKdf.	object	Section 4.5.1.1.1	Yes
oneStepNoCounterKdf	Indicates the IUT will be testing key derivation using the SP800-56Cr1 OneStepNoCounterKdf.	object	Section 4.5.1.1.2	Yes
twoStepKdf	Indicates the IUT will be testing key derivation using the SP800-56Cr1 TwoStepKdf.	object	Section 4.5.1.1.3	Yes

4.5.1.1.1. One Step KDF Capabilities

Table 7 — One Step KDF Options

JSON Value	Description	JSON Type	Valid Values	Optional
auxFunctions	The auxiliary functions to use with the KDF.	array of Table 8	See Table 8	No
fixedInfoPattern	The pattern used for fixedInfo construction.	string	See Section 4.5.1.3	No
encoding	The encoding type to use with fixedInfo construction. Note concatenation is currently supported. ASN.1 should be coming.	array of string	concatenation	No

Table 8 — AuxFunction Options

JSON Value	Description	JSON Type	Valid Values	Optional
auxFunctionName	The auxiliary function to use. Note that a customization string of “KDF” is used for the function when KMAC is utilized.	string	SHA-1, SHA2-224, SHA2-256, SHA2-384, SHA2-512, SHA2-512/224, SHA2-512/256, SHA3-224, SHA3-256, SHA3-384, SHA3-512, HMAC-SHA-1, HMAC-SHA2-224, HMAC-SHA2-256, HMAC-SHA2-384, HMAC-SHA2-512, HMAC-SHA2-512/224, HMAC-SHA2-512/256, HMAC-SHA3-224, HMAC-	No

JSON Value	Description	JSON Type	Valid Values	Optional
			SHA3-256, HMAC-SHA3-384, HMAC-SHA3-512, KMAC-128, KMAC-256	
macSaltMethods	How the salt is determined (default being all 00s, random being a random salt).	array of string	default, random	Not optional for mac based auxiliary functions.

4.5.1.1.2. One Step No Counter KDF Capabilities

The one step no counter KDF is a special implementation of the one step KDF. This implementation of the KDF does not utilize a 32 bit counter as a part of the concatenation that gets fed into function H. As such, there is no loop within the KDF due to there being no information changing between iterations of the potential concatenation, and the KDF output length is capped at the output length of the chosen aux function (or 2048 in the case of KMAC).

Table 9 — One Step No Counter KDF Options

JSON Value	Description	JSON Type	Valid Values	Optional
auxFunctions	The auxiliary functions to use with the KDF.	array of Table 10	See Table 10	No
fixedInfoPattern	The pattern used for fixedInfo construction.	string	See Section 4.5.1.3	No
encoding	The encoding type to use with fixedInfo construction. Note concatenation is currently supported. ASN.1 should be coming.	array of string	concatenation	No

Table 10 — AuxFunction Options

JSON Value	Description	JSON Type	Valid Values	Optional
auxFunctionName	The auxiliary function to use. Note that a customization string of “KDF”	string	SHA-1, SHA2-224, SHA2-256, SHA2-384, SHA2-512, SHA2-512/224, SHA2-512/256, SHA3-224,	No

JSON Value	Description	JSON Type	Valid Values	Optional
	is used for the function when KMAC is utilized.		SHA3-256, SHA3-384, SHA3-512, HMAC-SHA-1, HMAC-SHA2-224, HMAC-SHA2-256, HMAC-SHA2-384, HMAC-SHA2-512, HMAC-SHA2-512/224, HMAC-SHA2-512/256, HMAC-SHA3-224, HMAC-SHA3-256, HMAC-SHA3-384, HMAC-SHA3-512, KMAC-128, KMAC-256	
	The length of the keying material to derive (cannot exceed output length of aux function)	No	macSaltMethods	How the salt is determined (default being all 00s, random being a random salt).

4.5.1.1.3. Two Step KDF Capabilities

Table 11 — Two Step KDF Options

JSON Value	Description	JSON Type	Valid Values	Optional
capabilities	The capabilities supported for the Two Step KDF.	array of Table 12	See Table 12	No

Note this capabilities object is very similar to the capability object from SP800-108. Specific restrictions inherited from SP800-108 include the following.

- The ‘fixedDataOrder’ options “none” and “before iterator” are not valid for “counter” KDF. The ‘fixedDataOrder’ option “middle fixed data” is not valid for “feedback” nor “double pipeline iterator” KDF.
- A ‘counterLength’of 0 describes that there is no counter used. The 0 option is not valid for “counter” KDF.
- When’counterLength’ contains a value of “0”, ‘fixedDataOrder’must contain a value of “none” and vice versus.

Table 12 — TwoStepCapabilities Options

JSON Value	Description	JSON Type	Valid Values	Optional
macSaltMethod	How the salt is determined	array of string	default, random	Not optional for mac based

JSON Value	Description	JSON Type	Valid Values	Optional
	(default being all 00s, random being a random salt).			auxiliary functions.
fixedInfoPattern	The pattern used for fixedInfo construction.	string	See Section 4.5.1.3	No
encoding	The encoding type to use with fixedInfo construction. Note concatenation is currently supported. ASN.1 should be coming.	array of string	concatenation	No
kdfMode	The strategy for running the KDF.	string	counter, fedback, double pipeline iteration	No
macMode	The macMode supported by the KDF.	array of string	CMAC-AES128, CMAC-AES192, CMAC-AES256, HMAC-SHA-1, HMAC-SHA2-224, HMAC-SHA2-256, HMAC-SHA2-384, HMAC-SHA2-512, HMAC-SHA2-512/224, HMAC-SHA2-512/256, HMAC-SHA3-224, HMAC-SHA3-256, HMAC-SHA3-384, HMAC-SHA3-512	No
fixedDataOrder	The counter locations supported by the KDF.	array of string	none, before fixed data, after fixed data, before iterator	No
counterLength	The counter lengths supported for the KDF.	array of integer	8, 16, 24, 32	Not optional for counter mode.
supportedLengths	The supported derivation lengths.	domain	Single range (of literal) expected. Registered value must support the L value provided.	No

JSON Value	Description	JSON Type	Valid Values	Optional
supportsEmptyIv	The KDF supports an empty IV (feedback mode).	boolean	true, false	No
requiresEmptyIv	The KDF requires an empty IV (feedback mode).	boolean	true, false	Yes

4.5.1.2. Supported KeyConfirmation Method

Table 13 — KAS ECC KeyConfirmation Capabilities JSON Values

JSON Value	Description	JSON Type	Valid Values	Optional
macMethods	The MAC methods to use when testing KAS or KTS schemes with key confirmation.	object	Section 4.5.1.4	No
keyConfirmationDirections	The directions in which key confirmation is supported.	array	unilateral, bilateral	No
keyConfirmationRoles	The roles in which key confirmation is supported.	array	provider, recipient	No

4.5.1.3. FixedInfoPatternConstruction

IUTs **MUST** be capable of specifying how the FixedInfo is constructed for the KAS/KTS negotiation. Note that for the purposes of testing against the ACVP system, both uPartyInfo and vPartyInfo are **REQUIRED** to be registered within the fixed info pattern.

Pattern candidates:

- literal[0123456789ABCDEF]
  - uses the specified hex within "[]". literal[0123456789ABCDEF] substitutes "0123456789ABCDEF" in place of the field
- uPartyInfo
  - uPartyId { || ephemeralKey } { || ephemeralNonce } { || dkmNonce } { || c }
    - "optional" items such as ephemeralKey **MUST** be included when available for ACVP testing.

- vPartyInfo
  - vPartyId { || ephemeralKey } { || ephemeralNonce } { || dkmNonce } { || c }
    - “optional” items such as ephemeralKey **MUST** be included when available for ACVP testing.
- context
  - Random value chosen by ACVP server to represent the context.
- algorithmId
  - Random value chosen by ACVP server to represent the algorithmId.
- label
  - Random value chosen by ACVP server to represent the label.
- l
  - The length of the derived keying material in bits, **MUST** be represented in 32 bits for ACVP testing.

Example (Note that party U is the server in this case “434156536964”, party V is the IUT “a1b2c3d4e5”):

- “concatenation” : “literal[123456789CAFECAFE]||uPartyInfo||vPartyInfo”

Evaluated as:

- “123456789CAFECAFE434156536964a1b2c3d4e5”

4.5.1.4. Supported MAC Methods

Note that AT LEAST one mac method must be supplied when making use of Key Confirmation.

Table 14 — MAC Method Options

JSON Value	Description	JSON Type	Valid Values	Optional
CMAC	Utilizes CMAC as the MAC algorithm.	object	See Section 4.5.1.4.1. Note that the keyLen must be 128, 192, or 256 for this MAC.	Yes
HMAC-SHA-1	Utilizes HMAC-SHA-1 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA2-224	Utilizes HMAC-SHA2-224 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA2-256	Utilizes HMAC-SHA2-256 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA2-384	Utilizes HMAC-SHA2-384 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes

JSON Value	Description	JSON Type	Valid Values	Optional
HMAC-SHA2-512	Utilizes HMAC-SHA2-512 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA2-512/224	Utilizes HMAC-SHA2-512/224 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA2-512/256	Utilizes HMAC-SHA2-512/256 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA3-224	Utilizes HMAC-SHA3-224 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA3-256	Utilizes HMAC-SHA3-256 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA3-384	Utilizes HMAC-SHA3-384 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
HMAC-SHA3-512	Utilizes HMAC-SHA3-512 as the MAC algorithm.	object	See Section 4.5.1.4.1	Yes
KMAC-128	Utilizes KMAC-128 as the MAC algorithm. Note that a customization string of “KC” is used for the function when KMAC is utilized for Key Confirmation.	object	See Section 4.5.1.4.1	Yes
KMAC-256	Utilizes KMAC-256 as the MAC algorithm. Note that a customization string of “KC” is used for the function when KMAC is utilized for Key Confirmation.	object	See Section 4.5.1.4.1	Yes

4.5.1.4.1. Supported MAC Options

Table 15 — MAC Method Base Options

JSON Value	Description	JSON Type	Valid Values	Optional
keyLen	The amount of bits from the DKM to pass into the KeyConfirmation MAC function.	integer	128—512. Note that the DKM is **REQUIRED** to have at least 8 bits available after subtracting the keyLen specified.	No
macLen	The amount of bits to use as the tag from the MAC function.	integer	64—512.	No

4.6. Example KAS-ECC Registration

The following is a example JSON object advertising support for KAS ECC.

```
{
  "algorithm": "KAS-ECC",
  "revision": "Sp800-56Ar3",
  "prereqVals": [
    {
      "algorithm": "ECDSA",
      "valValue": "123456"
    },
    {
      "algorithm": "DRBG",
      "valValue": "123456"
    },
    {
      "algorithm": "SHA",
      "valValue": "123456"
    },
    {
      "algorithm": "KMAC",
      "valValue": "123456"
    },
    {
      "algorithm": "HMAC",
      "valValue": "123456"
    }
  ],
  "function": [
    "keyPairGen",
    "partialVal"
  ],
  "iutId": "123456ABCD",
  "scheme": {
    "ephemeralUnified": {
      "kasRole": [
        "initiator",
        "responder"
      ],
      "kdfMethods": {
        "oneStepKdf": {
          "auxFunctions": [
            {
              "auxFunctionName": "KMAC-128",
              "macSaltMethods": [
```

```
            "default"
          ]
        }
      ],
      "fixedInfoPattern": "algorithmId||l||uPartyInfo||vPartyInfo",
      "encoding": [
        "concatenation"
      ]
    },
    "oneStepNoCounterKdf": {
      "auxFunctions": [
        {
          "auxFunctionName": "KMAC-128",
          "l": 256,
          "macSaltMethods": [
            "default"
          ]
        }
      ],
      "fixedInfoPattern": "algorithmId||l||uPartyInfo||vPartyInfo",
      "encoding": [
        "concatenation"
      ]
    },
    "twoStepKdf": {
      "capabilities": [
        {
          "macSaltMethods": [
            "random"
          ],
          "fixedInfoPattern": "l||label||uPartyInfo||vPartyInfo||context",
          "encoding": [
            "concatenation"
          ],
          "kdfMode": "feedback",
          "macMode": [
            "HMAC-SHA3-224"
          ],
          "supportedLengths": [
            512
          ],
          "fixedDataOrder": [
            "after fixed data"
          ],
          "counterLength": [
            32
```

```
            ],
            "requiresEmptyIv": false,
            "supportsEmptyIv": false
          }
        ]
      }
    },
    "l": 512
  },
  "onePassDh": {
    "kasRole": [
      "initiator",
      "responder"
    ],
    "kdfMethods": {
      "oneStepKdf": {
        "auxFunctions": [
          {
            "auxFunctionName": "KMAC-128",
            "macSaltMethods": [
              "default"
            ]
          }
        ],
        "fixedInfoPattern": "algorithmId||l||uPartyInfo||vPartyInfo",
        "encoding": [
          "concatenation"
        ]
      },
      "twoStepKdf": {
        "capabilities": [
          {
            "macSaltMethods": [
              "random"
            ],
            "fixedInfoPattern": "l||label||uPartyInfo||vPartyInfo||context",
            "encoding": [
              "concatenation"
            ],
            "kdfMode": "feedback",
            "macMode": [
              "HMAC-SHA3-224"
            ],
            "supportedLengths": [
              512
            ],
```

```
            "fixedDataOrder": [
              "after fixed data"
            ],
            "counterLength": [
              32
            ],
            "requiresEmptyIv": false,
            "supportsEmptyIv": false
          }
        ]
      }
    },
    "keyConfirmationMethod": {
      "macMethods": {
        "KMAC-128": {
          "keyLen": 128,
          "macLen": 128
        }
      },
      "keyConfirmationDirections": [
        "unilateral"
      ],
      "keyConfirmationRoles": [
        "provider",
        "recipient"
      ]
    },
    "l": 512
  }
 },
 "domainParameterGenerationMethods": [
  "P-224"
 ]
}
```


Figure 2

5. Generation Requirements per Party per Scheme

The various schemes of KAS all have their own requirements as to keys and nonces per scheme, per party. The below table demonstrates those generation requirements:

Table 16 — Required Party Generation Obligations

Scheme	KasMode	KasRole	KeyConfirmationR	KeyConfirmationDirecti	StaticKeyPa	EphemeralKeyP	EphemeralNon	akmNon
fullUnified	NoKdfNoKc	InitiatorPartyV	None	None	True	True	False	False
fullUnified	NoKdfNoKc	ResponderPartyV	None	None	True	True	False	False
fullUnified	KdfNoKc	InitiatorPartyV	None	None	True	True	False	False
fullUnified	KdfNoKc	ResponderPartyV	None	None	True	True	False	False
fullUnified	KdfKc	InitiatorPartyV	Provider	Unilateral	True	True	False	False
fullUnified	KdfKc	InitiatorPartyV	Provider	Bilateral	True	True	False	False
fullUnified	KdfKc	InitiatorPartyV	Recipient	Unilateral	True	True	False	False
fullUnified	KdfKc	InitiatorPartyV	Recipient	Bilateral	True	True	False	False
fullUnified	KdfKc	ResponderPartyV	Provider	Unilateral	True	True	False	False
fullUnified	KdfKc	ResponderPartyV	Provider	Bilateral	True	True	False	False
fullUnified	KdfKc	ResponderPartyV	Recipient	Unilateral	True	True	False	False
fullUnified	KdfKc	ResponderPartyV	Recipient	Bilateral	True	True	False	False
fullMqv	NoKdfNoKc	InitiatorPartyV	None	None	True	True	False	False
fullMqv	NoKdfNoKc	ResponderPartyV	None	None	True	True	False	False
fullMqv	KdfNoKc	InitiatorPartyV	None	None	True	True	False	False
fullMqv	KdfNoKc	ResponderPartyV	None	None	True	True	False	False
fullMqv	KdfKc	InitiatorPartyV	Provider	Unilateral	True	True	False	False
fullMqv	KdfKc	InitiatorPartyV	Provider	Bilateral	True	True	False	False
fullMqv	KdfKc	InitiatorPartyV	Recipient	Unilateral	True	True	False	False
fullMqv	KdfKc	InitiatorPartyV	Recipient	Bilateral	True	True	False	False
fullMqv	KdfKc	ResponderPartyV	Provider	Unilateral	True	True	False	False
fullMqv	KdfKc	ResponderPartyV	Provider	Bilateral	True	True	False	False
fullMqv	KdfKc	ResponderPartyV	Recipient	Unilateral	True	True	False	False
fullMqv	KdfKc	ResponderPartyV	Recipient	Bilateral	True	True	False	False
ephemeralUnified	KdfNoKc	InitiatorPartyV	None	None	False	True	False	False
ephemeralUnified	KdfNoKc	ResponderPartyV	None	None	False	True	False	False
ephemeralUnified	KdfNoKc	InitiatorPartyV	None	None	False	True	False	False
ephemeralUnified	KdfNoKc	ResponderPartyV	None	None	False	True	False	False
onePassUnified	NoKdfNoKc	InitiatorPartyV	None	None	True	True	False	False
onePassUnified	NoKdfNoKc	ResponderPartyV	None	None	True	False	False	False
onePassUnified	KdfNoKc	InitiatorPartyV	None	None	True	True	False	False
onePassUnified	KdfNoKc	ResponderPartyV	None	None	True	False	False	False
onePassUnified	KdfKc	InitiatorPartyV	Provider	Unilateral	True	True	False	False
onePassUnified	KdfKc	InitiatorPartyV	Provider	Bilateral	True	True	False	False
onePassUnified	KdfKc	InitiatorPartyV	Recipient	Unilateral	True	True	False	False
onePassUnified	KdfKc	InitiatorPartyV	Recipient	Bilateral	True	True	False	False

Scheme	KasMode	KasRole	KeyConfirmationRole	KeyConfirmationDirecti	StaticKeyPa	EphemeralKeyP	EphemeralNonc	OtkmNonc
onePassUnified	KdfKc	ResponderPartyV	Provider	Unilateral	True	False	False	False
onePassUnified	KdfKc	ResponderPartyV	Provider	Bilateral	True	False	True	False
onePassUnified	KdfKc	ResponderPartyV	Recipient	Unilateral	True	False	True	False
onePassUnified	KdfKc	ResponderPartyV	Recipient	Bilateral	True	False	True	False
onePassMqv	NoKdfNoKc	InitiatorPartyU	None	None	True	True	False	False
onePassMqv	NoKdfNoKc	ResponderPartyV	None	None	True	False	False	False
onePassMqv	KdfNoKc	InitiatorPartyU	None	None	True	True	False	False
onePassMqv	KdfNoKc	ResponderPartyV	None	None	True	False	False	False
onePassMqv	KdfKc	InitiatorPartyU	Provider	Unilateral	True	True	False	False
onePassMqv	KdfKc	InitiatorPartyU	Provider	Bilateral	True	True	False	False
onePassMqv	KdfKc	InitiatorPartyU	Recipient	Unilateral	True	True	False	False
onePassMqv	KdfKc	InitiatorPartyU	Recipient	Bilateral	True	True	False	False
onePassMqv	KdfKc	ResponderPartyV	Provider	Unilateral	True	False	False	False
onePassMqv	KdfKc	ResponderPartyV	Provider	Bilateral	True	False	True	False
onePassMqv	KdfKc	ResponderPartyV	Recipient	Unilateral	True	False	True	False
onePassMqv	KdfKc	ResponderPartyV	Recipient	Bilateral	True	False	True	False
onePassDh	NoKdfNoKc	InitiatorPartyU	None	None	False	True	False	False
onePassDh	NoKdfNoKc	ResponderPartyV	None	None	True	False	False	False
onePassDh	KdfNoKc	InitiatorPartyU	None	None	False	True	False	False
onePassDh	KdfNoKc	ResponderPartyV	None	None	True	False	False	False
onePassDh	KdfKc	InitiatorPartyU	Recipient	Unilateral	False	True	False	False
onePassDh	KdfKc	ResponderPartyV	Provider	Unilateral	True	False	False	False
staticUnified	NoKdfNoKc	InitiatorPartyU	None	None	True	False	False	False
staticUnified	NoKdfNoKc	ResponderPartyV	None	None	True	False	False	False
staticUnified	KdfNoKc	InitiatorPartyU	None	None	True	False	False	True
staticUnified	KdfNoKc	ResponderPartyV	None	None	True	False	False	False
staticUnified	KdfKc	InitiatorPartyU	Provider	Unilateral	True	False	False	True
staticUnified	KdfKc	InitiatorPartyU	Provider	Bilateral	True	False	False	True
staticUnified	KdfKc	InitiatorPartyU	Recipient	Unilateral	True	False	False	True
staticUnified	KdfKc	InitiatorPartyU	Recipient	Bilateral	True	False	False	True
staticUnified	KdfKc	ResponderPartyV	Provider	Unilateral	True	False	False	False
staticUnified	KdfKc	ResponderPartyV	Provider	Bilateral	True	False	True	False
staticUnified	KdfKc	ResponderPartyV	Recipient	Unilateral	True	False	True	False
staticUnified	KdfKc	ResponderPartyV	Recipient	Bilateral	True	False	True	False

6. Test Vectors

The ACVP server provides test vectors to the ACVP client, which are then processed and returned to the ACVP server for validation. A typical ACVP validation test session would require multiple test vector sets to be downloaded and processed by the ACVP client. Each test vector set represents an individual algorithm defined during the capability exchange. This section describes the JSON schema for a test vector set used with SP800-56Ar3 KAS ECC algorithms.

The test vector set JSON schema is a multi-level hierarchy that contains meta data for the entire vector set as well as individual test vectors to be processed by the ACVP client. The following table describes the JSON elements at the top level of the hierarchy.

Table 17 — Top Level Test Vector JSON Elements

JSON Values	Description	JSON Type
acvVersion	Protocol version identifier	string
vsId	Unique numeric vector set identifier	integer
algorithm	Algorithm defined in the capability exchange	string
mode	Mode defined in the capability exchange	string
revision	Protocol test revision selected	string
testGroups	Array of test groups containing test data, see Section 6.1	array

An example of this would look like this

```
{
  "acvVersion": "version",
  "vsId": 1,
  "algorithm": "Alg1",
  "mode": "Mode1",
  "revision": "Revision1.0",
  "testGroups": [ ... ]
}
```

Figure 3

6.1. Test Groups JSON Schema

The testGroups element at the top level in the test vector JSON object is an array of test groups. Test vectors are grouped into similar test cases to reduce the amount of data transmitted in the vector set. For instance, all test vectors that use the same key size would be grouped together. The Test Group JSON object contains meta data that applies to all test vectors within the group. The following table describes the secure hash JSON elements of the Test Group JSON object.

The test group for KAS/KTS ECC is as follows:

Table 18 — Vector Group JSON Object

JSON Value	Description	JSON Type	Optional
tgId	Numeric identifier for the test group, unique across the entire vector set.	value	No
testType	The type of test for the group (AFT or VAL).	value	No
scheme	The scheme in use for the group. See Section 4.5.1 for possible values.	value	No
kasRole	The group role from the perspective of the IUT.	value	No
	The length of key to derive/transport.	value	No
iutId	The Iut's identifier.	value	No
serverId	The ACVP server's identifier.	value	No
kdfConfiguration	The KDF configuration for the group.	Object, See Section 6.1.1	No
macConfiguration	The MAC configuration for the group.	Object, See Section 6.1.2	Not optional for schemes using key confirmation.
keyConfirmationDirection	The key confirmation direction.	value	Yes
keyConfirmationRole	The key confirmation role.	value	Yes
domainParameterGenerationMode	The domain parameter type used.	value	No
tests	The tests for the group.	Array of objects, See Section 6.2.	No

6.1.1. KDF Configuration JSON Schema

Describes the KDF configuration for use under the test group.

Table 19 — KdfConfiguration JSON Object

JSON Value	Description	JSON Type	Optional
kdfType	The type of KDF to use for the group.	value — oneStep, oneStepNoCounter, twoStep	No
saltMethod	The strategy used for salting.	value — default (all 00s), random	No
fixedInfoPattern	The pattern used for constructing the fixedInfo.	value — See Section 4.5.1.3.	No
fixedInfoEncoding	The pattern used for constructing the fixedInfo.	value — See Section 4.5.1.3.	No
auxFunction	The auxiliary function used in the KDF.	value — See Table 8.	Not optional for OneStepKdf.
macMode	The MAC function used in the KDF.	value — See Table 12.	Not optional for TwoStepKdf.
counterLocation	The counter location.	value	Yes
counterLen	The counter length.	value	Yes
ivLen	The iv length.	value	Yes

6.1.2. MAC Configuration JSON Schema

Describes the key confirmation MAC configuration for use under the test group.

Table 20 — MacConfiguration JSON Object

JSON Value	Description	JSON Type	Optional
macType	The macType used in key confirmation.	value — HMAC-SHA2-224, HMAC-SHA2-256, HMAC-SHA2-384, HMAC-SHA2-512, HMAC-SHA2-512/224, HMAC-SHA2-512/256, HMAC-SHA3-224, HMAC-SHA3-256, HMAC-SHA3-384, HMAC-SHA3-512, CMAC, KMAC-128, KMAC-256	No

JSON Value	Description	JSON Type	Optional
keyLen	The number of bits to take from the DKM to use for the mac key in key confirmation.	value	No
macLen	The number of bits to use for the MAC tag.	value	No

6.2. Test Case JSON Schema

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each KAS/KTS ECC test vector.

Table 21 — Test Case JSON Object

JSON Value	Description	JSON Type	Optional
tcId	Numeric identifier for the test case, unique across the entire vector set.	value	No
ephemeralPublicKeyIutX	The IUT's ephemeral public key X value.	value	Yes
ephemeralPublicKeyIutY	The IUT's ephemeral public key Y value.	value	Yes
staticPublicKeyIutX	The IUT's static public key X value.	value	Yes
staticPublicKeyIutY	The IUT's static public key Y value.	value	Yes
ephemeralPublicKeyServerX	The Server's ephemeral public key X value.	value	Yes
ephemeralPublicKeyServerY	The Server's ephemeral public key Y value.	value	Yes
staticPublicKeyServerX	The Server's static public key X value.	value	Yes
staticPublicKeyServerY	The Server's static public key Y value.	value	Yes
dkmNonceIut	The IUT's nonce used in static schemes for Key Confirmation.	value	Yes
ephemeralNonceIut	The IUT's ephemeral nonce used in some schemes.	value	Yes
dkmNonceServer	The Server's nonce used in static	value	Yes

JSON Value	Description	JSON Type	Optional
	schemes for Key Confirmation.		
ephemeralNonceServer	The Server's ephemeral nonce used in some schemes.	value	Yes
staticPrivateKeyIut	The IUT's static private key.	value	Yes
ephemeralPrivateKeyIut	The IUT's ephemeral private key.	value	Yes
kdfParameter	The KDF parameters for this test case.	value — See Section 6.2.1.	Yes
dkm	The derived keying material.	value	Yes
tag	The tag generated as a part of key conformation (from the IUT perspective).	value	Yes

6.2.1. KDF Parameter JSON Schema

KDF specific options used for the test case.

Table 22 — KDF Parameter JSON Object

JSON Value	Description	JSON Type	Optional
kdfType	The type of KDF utilized.	value	No
salt	The salt used for the test case.	value	Yes
iv	The iv used for the test case.	value	Yes
algorithmId	The random "algorithID" used for the test case when applicable to the fixedInfo pattern.	value	Yes
context	The random "context" used for the test case when applicable to the fixedInfo pattern.	value	Yes
label	The random "label" used for the test case when applicable to the fixedInfo pattern.	value	Yes

6.3. Example Test Vectors JSON Object KAS-FFC

The following is a example JSON object for KAS-FFC test vectors sent from the ACVP server to the crypto module.


```
{
  "vsId": 0,
  "algorithm": "KAS-ECC",
  "revision": "Sp800-56Ar3",
  "testGroups": [
    {
      "tgId": 1,
      "testType": "AFT",
      "tests": [
        {
          "staticPublicServerX":
 "B7A4DDA5DC3A317647B39F39E05390A88F12F53861C24635",
          "staticPublicServerY":
 "CA2776BF6A0F35B727F3057340E89A1600915B81BB2E87B7",
          "tcId": 1,
          "ephemeralNonceServer":
 "44588073AACC3CFD6C9A5E2A0973B6BDDFC35F67EEA96FD0B070DF05F24A4B381F05CE9ACC67739B157CF8EE
          "kdfParameter": {
            "kdfType": "oneStep",
            "salt": "00000000000000000000000000000000",
            "algorithmId": "A51CF275ABE573209CBC606A934352FE"
          }
        }
      ],
      "domainParameterGenerationMode": "P-224",
      "scheme": "staticUnified",
      "kasRole": "initiator",
      "l": 512,
      "iutId": "123456ABCD",
      "serverId": "434156536964",
      "kdfConfiguration": {
        "kdfType": "oneStep",
        "saltMethod": "default",
        "fixedInfoPattern": "algorithmId||l||uPartyInfo||vPartyInfo",
        "fixedInfoEncoding": "concatenation",
        "auxFunction": "KMAC-128"
      },
      "macConfiguration": {
        "macType": "KMAC-128",
        "keyLen": 128,
        "macLen": 128
```

```
      },
      "keyConfirmationDirection": "unilateral",
      "keyConfirmationRole": "provider"
    },
    {
      "tgId": 2,
      "testType": "VAL",
      "tests": [
        {
          "staticPublicServerX":
"87F6D507656EBC3D4D655FD4C0F13BE0F98D5B7472A3B247",
          "staticPublicServerY":
"CFBC8EE38F4EF2DF1B97BF410ABCF4968F1115E7B80E34C6",
          "staticPrivateIut":
"F43B6F08F570D469ED31CF920516114B1B5E3C3C7BDD6B14",
          "staticPublicIutX":
"7573E06C6BACA56D5AFD08A1A014776BDDA7F4593645A07D",
          "staticPublicIutY":
"93D0C1CDC5C23BD045AD6258448436A55E3C310B4333F551",
          "tcId": 21,
          "ephemeralNonceServer":
"6F4C587D3CEF0B1D0D5B359B18FFB8B72C879EB3997E768826552082D56931D965E7F315FD7254C434871FA1
          "dkmNonceIut":
"AB5CCC3B75AA1FB85D28D5D53126B362AAABA3C51D427B6D138BEFD7EE636E1BC239FB45630BF6D7F0E80B59
          "kdfParameter": {
            "kdfType": "oneStep",
            "salt": "00000000000000000000000000000000",
            "algorithmId": "342BCBC9DE15458BCA294BD16FFA10A7"
          },
          "dkm":
"B9FDC93EA0B6A7906C6DB8EC17475B3073A8AD1C24CB1287AB8A6AEA46CABA4FDFD7B0CB77F74CDCF3DFF8DC
          "tag": "3279D63C9192B7FEF71F6735921B3B46"
        }
      ],
      "domainParameterGenerationMode": "P-224",
      "scheme": "staticUnified",
      "kasRole": "initiator",
      "l": 512,
      "iutId": "123456ABCD",
      "serverId": "434156536964",
      "kdfConfiguration": {
        "kdfType": "oneStep",
        "saltMethod": "default",
        "fixedInfoPattern": "algorithmId||l||uPartyInfo||vPartyInfo",
        "fixedInfoEncoding": "concatenation",
        "auxFunction": "KMAC-128"
```

```
      },
      "macConfiguration": {
        "macType": "KMAC-128",
        "keyLen": 128,
        "macLen": 128
      },
      "keyConfirmationDirection": "unilateral",
      "keyConfirmationRole": "provider"
    }
  ]
}
```

Figure 4

7. Test Vector Responses

After the ACVP client downloads and processes a vector set, it **MUST** send the response vectors back to the ACVP server. The following table describes the JSON object that represents a vector set response.

Table 23 — Vector Set Response JSON Object

JSON Value	Description	JSON type	Optional
acvVersion	Protocol version identifier	value	No
vsId	Unique numeric identifier for the vector set	value	No
testGroups	Array of JSON objects that represent each test vector group. See Table 24.	array	No

The testGroups section is used to organize the ACVP client response in a similar manner to how it receives vectors. Several algorithms **SHALL** require the client to send back group level properties in their response. This structure helps accommodate that.

Table 24 — Vector Set Group Response JSON Object

JSON Value	Description	JSON type	Optional
tgId	The test group Id	value	No
tests	Array of JSON objects that represent each test vector group. See Table 25.	array	No

The testCase section is used to organize the ACVP client response in a similar manner to how it receives vectors. Several algorithms **SHALL** require the client to send back group level properties in their response. This structure helps accommodate that.

Table 25 — Vector Set Test Case Response JSON Object

JSON Value	Description	JSON type	Optional
tcId	The test case Id	value	No
testPassed	Used in VAL test types, should the KAS/KTS negotiation have succeeded?	boolean	Yes
ephemeralPublicKeyIutX	The IUT's ephemeral public key X value.	value	Yes
ephemeralPublicKeyIutY	The IUT's ephemeral public key Y value.	value	Yes
staticPublicKeyIutX	The IUT's static public key X value.	value	Yes

JSON Value	Description	JSON type	Optional
staticPublicKeyIutX	The IUT's static public key Y value.	value	Yes
dkmNonceIut	The IUT's nonce used in static schemes for Key Confirmation.	value	Yes
ephemeralNonceIut	The IUT's ephemeral nonce used in some schemes.	value	Yes
dkm	The derived keying material.	value	Yes
tag	The tag generated as a part of key confirmation (from the IUT perspective).	value	Yes

7.1. Example Test Results KAS-ECC JSON Object

The following is an example JSON object for KAS-ECC test results sent from the crypto module to the ACVP server.

```
[
  {
    "acvVersion": "version"
  },
  {
    "vsId": 0,
    "algorithm": "KAS-ECC",
    "revision": "Sp800-56Ar3",
    "testGroups": [
      {
        "tgId": 1,
        "tests": [
          {
            "staticPublicIutX":
 "ED9CF3FE1B79D014F7FF60DFDBFC19457C4F3EBEB0BB10B5",
            "staticPublicIutY":
 "5CA8819BC0D39E67AE9AB4747DC563ADA1AE1E9DBA12C272",
            "tcId": 1,
            "dkmNonceIut":
 "215D9AB3A371B395802FD0FCD97815EDFC468DC631735BAEEA0F18498EFC3B52BBABD2B953DE7B64EF20D899
            "dkm":
 "56505307C7F11F4640C96D863FA3634120F2B2CAB262AE29B1CD26252BC1537E84DF3EB75C1E240983B599B3
            "tag": "CE39683069F0DA7624F72086FB4B2B8E"
          }
        ]
```

```
      },
      {
        "tgId": 3,
        "tests": [
          {
            "tcId": 21,
            "testPassed": true
          }
        ]
      }
    ]
  }
]
```

Figure 5

8. ECC CDH Component Test

The ECC CDH Component Test for SP800-56Ar3

8.1. ECC CDH Component Capabilities JSON Values

Each algorithm capability advertised is a self-contained JSON object using the following values.

Table 26 — KAS ECC Component Capabilities JSON Values

JSON Value	Description	JSON type	Valid Values	Optional
algorithm	The algorithm under test	value	KAS-ECC	No
mode	The algorithm mode	value	CDH-Component	No
revision	The algorithm testing revision to use.	value	“Sp800-56Ar3”	No
prereqVals	Prerequisite algorithm validations	array of prereqAlgVal objects	See Section 4.2	No
function	Type of function supported	array	See Section 4.4	Yes
curve	Array of supported curves	array	P-224, P-256, P-384, P-521, K-233, K-283, K-409, K-571, B-233, B-283, B-409, B-571	No

8.1.1. Example KAS ECC CDH-Component Capabilities JSON Object

The following is a example JSON object advertising support for KAS ECC CDH-Component.

```
{
 "algorithm": "KAS-ECC",
 "mode": "CDH-Component",
 "revision": "Sp800-56Ar3",
 "prereqVals": [{
  "algorithm": "ECDSA",
  "valValue": "123456"
 }],
 "function": ["keyPairGen"],
 "curve": ["P-224", "K-233", "B-233"]
```

}

Figure 6

8.2. ECC CDH Component TestVectors JSON Values

Table 27 — KAS ECC CDH Component TestVectors JSON Values

JSON Value	Description	JSON type	Valid Values	Optional
algorithm	The algorithm under test	value	KAS-ECC	No
mode	The algorithm mode under test	value	CDH-Component	No
revision	The algorithm testing revision to use.	value	“Sp800-56Ar3”	No
testGroups	Array of individual test group JSON objects, which are defined in Section 8.2.1	Array	Array of test group information	No

8.2.1. ECC CDH Component TestGroup JSON Values

Table 28 — KAS ECC CDH Component TestGroup JSON Values

JSON Value	Description	JSON type	Valid Values	Optional
testType	The test type expected within the group. AFT is the only valid value for ECC Component.	value	AFT	No
curve	The curve used in the test group	value	P-224, P-256, P-384, P-521, K-233, K-283, K-409, K-571, B-233, B-283, B-409, B-571	No
tests	Array of individual test vector JSON objects, which are defined in Section 8.2.2	array		No

8.2.2. ECC CDH Component TestCase JSON Values

Table 29 — KAS ECC CDH Component TestCase JSON Values

JSON Value	Description	Valid Values	Optional
tcId	Numeric identifier for the test case, unique across the entire vector set.	value	No

JSON Value	Description	Valid Values	Optional
publicServerX	The X coordinate of the server's public key	value	Yes
publicServerY	The Y coordinate of the server's public key	value	Yes
publicIutX	The X coordinate of the iut's public key	value	No
publicIutY	The Y coordinate of the iut's public key	value	No
	The shared secret Z	value	No

8.2.3. Example KAS ECC CDH-Component Test Vectors JSON Object

The following is a example JSON object for KAS ECC CDH-Component test vectors sent from the ACVP server to the crypto module.

```
[{
  "acvVersion": "1.0"
 },
 {
  "vsId": 1750,
  "algorithm": "KAS-ECC",
  "mode": "CDH-Component",
  "revision": "Sp800-56Ar3",
  "testGroups": [{
    "tgId": 1,
    "testType": "AFT",
    "curve": "P-192",
    "tests": [{
     "tcId": 1,
     "publicServerX": "CAEF2CBA796BB7FC143D3EAED698C26AAE6F6F79DF3974EE",
     "publicServerY": "03ED6D7A90637629DBCEBFF4A2D1D771D9D4CF9F0D88CE90"
    }]
   },
   {
    "tgId": 2,
    "testType": "AFT",
    "curve": "K-163",
    "tests": [{
     "tcId": 26,
     "publicServerX": "048C46D674E1218D0BD3C9FCD120ECE8B4DB7310E7",
     "publicServerY": "ED3EEDB656E035C779081090BE44B743E857E3B4"
    }]
   },
   {
    "tgId": 3,
```

```
    "testType": "AFT",
    "curve": "B-163",
    "tests": [{
     "tcId": 51,
     "publicServerX": "8EE7C8F08BF47B21CA2FE911B721651B90E52391",
     "publicServerY": "0461DF3646E95598EAE4F5C6A634E71006ABC6FE1F"
    }]
   }
  ]
 }
]
```

Figure 7

8.3. KAS CDH-Component Test Vector Responses

After the ACVP client downloads and processes a vector set, it must send the response vectors back to the ACVP server. The following table describes the JSON object that represents a vector set response.

8.3.1. CDH Component Vector Set Response JSON Object

Table 30 — CDH Component Vector Set Response JSON Object

JSON Value	Description	JSON type
acvVersion	Protocol version identifier	value
vsId	Unique numeric identifier for the vector set	value
testGroups	Array of JSON objects that represent each test vector group. See Section 8.3.2	array

The testGroups section is used to organize the ACVP client response in a similar manner to how it receives vectors. Several algorithms **SHALL** require the client to send back group level properties in their response. This structure helps accommodate that.

8.3.2. CDH Component Vector Set Group Response JSON Object

Table 31 — CDH Component Vector Set Group Response JSON Object

JSON Value	Description	JSON type
tgId	The test group Id	value tests

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each DRBG test vector.

8.3.3. CDH Component Test Case Results JSON Object

Table 32 — CDH Component Test Case Results JSON Object

JSON Value	Description	JSON type	Optional
tcId	Numeric identifier for the test case, unique across the entire vector set.	value	No
publicIutX	x value of the IUT public key	value	No
publicIutY	x value of the IUT public key	value	No
	Computed shared secret Z	value	No

8.4. Example KAS ECC CDH Component Test Results JSON Object

The following is a example JSON object for KAS ECC CDH Component test results sent from the crypto module to the ACVP server.


```
[{
  "acvVersion": "1.0"
 },
 {
  "vsId": 1750,
  "testGroups": [{
    "tgId": 1,
    "tests": [{
     "tcId": 1,
     "publicIutX": "DB9FBC84CBAD3EED42C31CDBF2882041634D040219C3E47A",
     "publicIutY": "9BD672733BCCEF2BD805E97FF9BBFE0FFC003BEEEF56868B",
     "z": "8BEAEA60DFAC075F9F25A5CFEA39818D98D3EA4B9D4C34A8"
    }]
   },
   {
    "tgId": 2,
    "tests": [{
     "tcId": 26,
     "publicIutX": "058C593D1D4E8238102BDE6B497218D92F8EDD2997",
     "publicIutY": "0437682E4608984EFC7FB619FB260EF27CAF704D7B",
     "z": "075D9A831E0665521D613AEAA59B8C8CDFBAC8C683"
    }]
   },
   {
    "tgId": 3,
    "tests": [{
     "tcId": 51,
     "publicIutX": "04128CD094F6988AA26DA2B100A71A31214CC9C50B",
     "publicIutY": "01A3A88C9F0987E488922573D0A31D300532F0B268",
     "z": "07EC896621BF1703EB7567196ED1DE5742C4695990"
```

```
     }]
    }
   ]
  }
]
```

Figure 8

9. Security Considerations

There are no additional security considerations outside of those outlined in the ACVP document.

Appendix A — Terminology

For the purposes of this document, the following terms and definitions apply.

A.1.

Prompt
JSON sent from the server to the client describing the tests the client performs
Registration
The initial request from the client to the server describing the capabilities of one or several algorithm, mode and revision combinations
Response
JSON sent from the client to the server in response to the prompt
Test Case
An individual unit of work within a prompt or response
Test Group
A collection of test cases that share similar properties within a prompt or response
Test Vector Set
A collection of test groups under a specific algorithm, mode, and revision
Validation
JSON sent from the server to the client that specifies the correctness of the response

Appendix B — Abbreviations and Acronyms

ACVP	Automated Crypto Validation Protocol
JSON	Javascript Object Notation

Appendix C — Revision History

Table C-1

Version	Release Date	Updates
1	2020-01-31	Initial Release

Appendix D — References

S. Bradner (March 1997) *Key words for use in RFCs to Indicate Requirement Levels* (Internet Engineering Task Force), BCP 14, March 1997. RFC 2119. RFC RFC2119. DOI 10.17487/RFC2119. https://www.rfc-editor.org/info/rfc2119.

T. Kivinen, M. Kojo (May 2003) *More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)* (Internet Engineering Task Force), RFC 3526, May 2003. RFC 3526. RFC RFC3526. DOI 10.17487/RFC3526. https://www.rfc-editor.org/info/rfc3526.

D. Gillmor (August 2016) *Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS)* (Internet Engineering Task Force), RFC 7919, August 2016. RFC 7919. RFC RFC7919. DOI 10.17487/RFC7919. https://www.rfc-editor.org/info/rfc7919.

P. Hoffman (December 2016) *The "xml2rfc" Version 3 Vocabulary* (Internet Engineering Task Force), RFC 7991, December 2016. RFC 7991. RFC RFC7991. DOI 10.17487/RFC7991. https://www.rfc-editor.org/info/rfc7991.

B. Leiba (May 2017) *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words* (Internet Engineering Task Force), BCP 14, May 2017. RFC 8174. RFC RFC8174. DOI 10.17487/RFC8174. https://www.rfc-editor.org/info/rfc8174.

National Institute of Standards and Technology (July 2013) *Digital Signature Standard (DSS)* (Gaithersburg, MD), July 2013. FIPS 186-4. https://doi.org/10.6028/NIST.FIPS.186-4.

Lily Chen (October 2009) *Recommendation for Key Derivation Using Pseudorandom Functions (Revised)* (Gaithersburg, MD), October 2009. SP 800-108. https://doi.org/10.6028/NIST.SP.800-108.

Elaine B. Barker, Lily Chen, Allen Roginsky, Apostol Vassilev, Richard Davis (April 2018) *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography* (Gaithersburg, MD), April 2018. SP 800-56A Rev. 3. https://doi.org/10.6028/NIST.SP.800-56Ar3.

Elaine B. Barker, Lily Chen, Richard Davis (April 2018) *Recommendation for Key-Derivation Methods in Key-Establishment Schemes* (Gaithersburg, MD), April 2018. SP 800-56C Rev. 1. https://doi.org/10.6028/NIST.SP.800-56Cr1.

Fussell B, Vassilev A, Booth H, Celi C, Hammett R (July 01, 2019) *Automatic Cryptographic Validation Protocol* (National Institute of Standards and Technology, Gaithersburg, MD), July 01, 2019.